

Technische Universität München

Mathematics preparatory course

2026

Dr. Michael Luttenberger – Jeremias Bohn

Remark: some commonly used abbreviations

- "i.e." is short for "id est" which is Latin and has to be read as "that is", "this means".
- "e.g." is short for "exempli gratia" which is again Latin and has to be read as "for the sake of an example", "for example", "for instance".
- "n.b." or "N.B." is short for "nota bene" which of course is also Latin and means "take special note", "observe!", "read carefully".

1 Introduction to logic

1.1 Truth tables and syntax trees

Consider the following "propositional formulas"

$$\begin{aligned} F_1 &= ((A \text{ and} (\text{if } B \text{ then } C)) \text{ or} (C \text{ and not } A)) \\ F_2 &= ((A \text{ and not } B) \text{ or} ((C \text{ or } B) \text{ and} (C \text{ or not } B))) \\ F_3 &= (((\text{not } A) \text{ iff } B) \text{ and} (B \text{ and} (A \text{ or } C))) \\ F_4 &= (((\text{not } A) \text{ and} (\text{if } C \text{ then } B)) \text{ and} (A \text{ or } C)) \end{aligned}$$

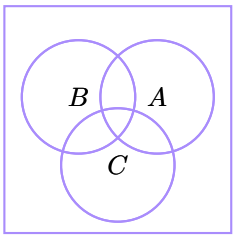
where A, B and C are placeholders (variables) for arbitrary propositions.

As each (unknown) propositions A, B, C can only be true or false, we can simply build a truth table (as done in the lecture) and consider all $2^3 = 8$ (hypothetical) situations (cases) – see (a).

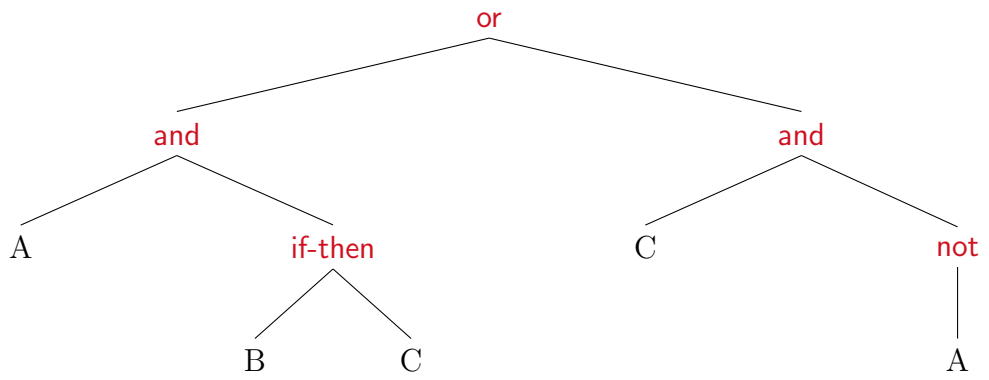
Recall that just as in case of arithmetic expressions, brackets indicate in which order to evaluate the connectives: e.g. given $A \text{ and} (B \text{ or } C)$ you first need to determine the truth value of $(B \text{ or } C)$, before you can determine the truth value of $A \text{ and} (B \text{ or } C)$ where you essentially replace $(B \text{ or } C)$ by its truth value.

(a) For each of the given propositions determine its truth table and also the corresponding Venn diagram. Use the following as templates (you may abbreviate true to t and false to f):

A	B	C	
false	false	false	
false	false	true	
false	true	false	
false	true	true	
true	false	false	
true	false	true	
true	true	false	
true	true	true	



(b) A *syntax tree* is a graphical representation of how a formalized proposition has to be "read" (evaluated). For example, the syntax tree of F_1 is:



Try to understand how the brackets of F_1 determine the structure of its syntax tree. Then also determine the syntax trees for the remaining propositions.

(In general, brackets are used to structure text by essentially encoding such a tree that guides the "reader". E.g. most compilers take the program code, check that the brackets match, and then represent the text internally as such a syntax tree. As another typical example: in XML and HTML opening and closing tags like `<b></b>` act as brackets.)

1.2 Brackets and ambiguity

(a) Brackets in general are used to resolve (syntactical) ambiguities (also in programming languages): For arithemtic expressions we usually have some sort of "operator precedence" e.g. "evaluate multiplication before addition":

$$(5 \cdot 3) + 3 = 5 \cdot 3 + 3 \neq 5 \cdot (3 + 3)$$

Similar precedence rules exist for propositional logic, but these often depend on the author/lecture/book/programming language.

Consider the following formalized proposition:

$$\text{not } A \text{ and not } B \text{ or } C$$

Without precedence rules, we could read it as

$$\text{not}(A \text{ and}((\text{not } B) \text{ or } C))$$

but also as

$$((\text{not } A) \text{ and}(\text{not}(B \text{ or } C)))$$

Enumerate all possible ways of how to structure

$$\text{not } A \text{ and not } B \text{ or } C$$

using brackets.

Afterwards decide which of the obtained expressions do not have the same meaning (e.g. using a truth table). E.g. you can use `www.wolframalpha.com` to obtain a truth table: simply enter "not$(A$ and$((\text{not } B)$ or $C))$".

Also see how wolframalpha decides to introduce brackets into "not A and not B or C".

(Remark: Wolframalpha will also output the formal representation of propositional formulas with $\neg$ for not, $\wedge$ for and, and $\vee$ for or; these are NOT to be used as abbreviations, but for propositions on propositions like in "the proposition $(A \wedge \neg A)$ is never true".)

(b) In *discrete structures* we will only make use of the single precedence rule:

"Negation (not) always binds the strongest"

What is the answer to (a) now? (I.e. how many syntax trees are "ruled out" by this precedence rule?)

1.3 Truth values versus numbers

Some programming languages (e.g. C/C++, Python) treat the truth values true and false as the natural numbers 1 and 0, respectively, but others (e.g. Java, ADA, Rust) don't as formally there is a difference between the concepts of "truth" and "number", and mixing both concepts can lead to subtle programming errors.

While treating "true" as 1 and "false" as 0 is *a reasonable* choice, it is not the only choice:

(i) Assume we *define* "true := 1" and "false := 0".

Check that then "and" coincides with "min", "or" coincides with "max", and "not" with "1−". E.g. simply complete the following truth/value table:

A	B	not A	A and B	A or B	A	B	$1-A$	$\max(A,B)$	$\min(A,B)$
false	false				0	0			
false	true				0	1			
true	false				1	0			
true	true				1	1			

(ii) For comparison, assume now we *define* "true := 0" and "false := 1".

How are the standard connectives "and", "or" and "not" now related to "min", "max" and "1−"?

(iii) Assume again that we *define* "true := 1" and "false := 0".

Give an alternative definition of and, or, and not using only $+$, $\cdot$, $-$ and 1.

(Perhaps you have seen in school already that $\Pr[A \cap B] = \Pr[A] + \Pr[B] - \Pr[A \cap B]$ and that $\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$ if A, B are stochastically independent.)

(iv) Assume now that we *define* "true := 1" and "false := −1".

Give an alternative definition of and, or, and not using only $+$, $\cdot$, $-$ and 1.

The main point is that there is no unique way of treating truth values as numbers, and thus it is best to distinguish truth values from numbers in general.

(i), (ii), (iii) allow also to model "confidence", "trust", "believe", "plausibility" in computer science by allowing values between 0 and 1 (e.g. fuzzy logic, information theory, machine learning).

1.4 NOR and NAND

One can show that for propositional logic the connectives "and", "or" and "not" suffice in the sense that any other connective can be expressed using these three standard connectives. e.g. "if A then B" is synonymous with (semantically equivalent to) "B or not A" (*independent of the concrete choice of* A, B).

CPUs are in fact described using propositional logic, and transistors are used to physically realize logical connectives.

But often not the three standard connectives are realized, but instead the connectives "nand" and "nor" are physically realized. The two connectives are defined by the following truth table:

A	B	$(A \text{ nand } B)$	$(A \text{ nor } B)$
false	false	true	true
false	true	true	false
true	false	true	false
true	true	false	false

In words: "A nand B" can be considered as an abbreviation (macro) of "not$(A$ and $B)$", and "A nor B" as an abbreviation of "not$(A$ or $B)$".

Express "and", "or" and "not" using only "nand" resp. only using "nor".

E.g. "not A" can be expressed as "$(A$ nand $A)$" resp. "$(A$ nor $A)$"

1.5 XOR

In formal logic, mathematics and programming languages "or" (disjunction) has the meaning of "at least one".

But in natural language "or" is often used in the sense of "either-or", i.e. "exactly one out of two" (exclusive alternative: choose exactly one).

The latter is typically called *exclusive-or* in computer science and often abbreviated to "xor" (also: "eXor"). We can define xor formally using again a truth table:

A	B	A or B	A xor B
false	false	false	false
false	true	true	true
true	false	true	true
true	true	true	false

Express "xor" using only the standard connectives "not", "and" and "or".

1.6 Syllogisms

Decide whether the following syllogisms are valid, i.e. that in every situation (independent of the precise meaning of the predicates), in which both statements above the dividing line (premises) are true, the statement underneath the dividing line (conclusion, consequence) is also *necessarily* true.

Visualize the relationship between the types of objects graphically using Venn/Euler diagrams as done in the slides. For each example also provide the first-order logic structure of each statement by "abstracting" from the concrete property by means of a generic "predicate symbol" as shown here:

$$\frac{\begin{array}{l}\text{All bandersnatches are borogoves}\\ \text{All borogoves are slithy}\end{array}}{\text{All bandersnatches are slithy}}$$

has the structure

$$\frac{\begin{array}{l}\mathsf{for\ all}\, x\colon \mathsf{if}\, P(x)\, \mathsf{then}\, Q(x)\\ \mathsf{for\ all}\, x\colon \mathsf{if}\, Q(x)\, \mathsf{then}\, R(x)\end{array}}{\mathsf{for\ all}\, x\colon \mathsf{if}\, P(x)\, \mathsf{then}\, R(x)}$$

where

$$\begin{array}{l}P(x) := x \text{ is a bandersnatch}\\ Q(x) := x \text{ is a borogove}\\ R(x) := x \text{ is slithy}\end{array}$$

Then group the syllogisms by their respective first-order logic structure.

Observe that the order of the premisses does not matter.

$$\frac{\begin{array}{l}\text{All rectangles are quadrilaterals}\\ \text{All squares are rectangles}\end{array}}{\text{All squares are quadrilaterals}} \qquad \frac{\begin{array}{l}\text{No rectangle is a circle}\\ \text{All squares are rectangles}\end{array}}{\text{Some squares are not circles}}$$

$$\frac{\begin{array}{l}\text{No rectangle is a circle}\\ \text{All squares are rectangles}\end{array}}{\text{No square is a circle}} \qquad \frac{\begin{array}{l}\text{All professors are serious}\\ \text{Some lecturers are not serious}\end{array}}{\text{Some lecturers are not professors}}$$

$$\frac{\begin{array}{l}\text{Some rhombi are rectangles}\\ \text{All squares are rectangles}\end{array}}{\text{Some rhombi are squares}} \qquad \frac{\begin{array}{l}\text{No mammal breathes through gills}\\ \text{All fish breathe through gills}\end{array}}{\text{No fish is a mammal}}$$

$$\frac{\begin{array}{l}\text{Some rhombi are squares}\\ \text{All squares are rectangles}\end{array}}{\text{Some rhombi are rectangles}} \qquad \frac{\begin{array}{l}\text{No mammal breathes through gills}\\ \text{All fish breathe through gills}\end{array}}{\text{No mammal is a fish}}$$

$$\frac{\begin{array}{l}\text{All squares are rectangles}\\ \text{Some rhombi are squares}\end{array}}{\text{Some rhombi are rectangles}} \qquad \frac{\begin{array}{l}\text{No animal breathing through gills is a mammal}\\ \text{Some marine animals are mammals}\end{array}}{\text{Some marine animals don't breath through gills}}$$

$$\frac{\begin{array}{l}\text{All rectangles are quadrilaterals}\\ \text{All squares are rectangles}\end{array}}{\text{Some squares are quadrilaterals}} \qquad \frac{\begin{array}{l}\text{Some fruits are apples}\\ \text{All fruits are parts of plants}\end{array}}{\text{Some parts of plants are apples}}$$

1.7 Inferences

Show that the following inferences are indeed valid:

if A then B A ... B (modus ponens)	if A then B not B ... not A (modus tollens)	A or B not B ... A	if A then B if B then C ... if A then C (resolution)

A or B if A then C if B then C ... C (case distinction)	if not B then not A ... if A then B (proof by contraposition)	if not A then B if not A then not B ... A (proof by contradiction)

All x with $P(x)$ are $Q(x)$ $P(d)$ ... $Q(d)$	All x with $P(x)$ are $Q(x)$ Some x are $P(x)$ ... Some x are $Q(x)$	All x with $P(x)$ are $Q(x)$ All x with $Q(x)$ are $R(x)$ ... All x with $P(x)$ are $R(x)$

For the first six, use e.g. a truth table w.r.t. the propositional variables A, B, C.

For the last three, use Euler diagrams. (The d denotes simply some*d*ing with property P.)

Alternatively argue as in the slides.

1.8 Syntax versus semantics (and De Moran's laws)

Consider the two propositional formulas

- not(A and B)
- (not A) or(not B)

"Syntactically" they are different: their syntax trees are different, they differ in the connectives that occur in them, they are simply different texts.

Yet "semantically" they are the "same" at least in the context of logic: their meaning is the same in every possible situation, their truth tables coincide.

(a) Draw the syntax trees of the two formulas.

Check that the two statements are semantically equivalent by determining the truth tables (and, if you like, using Venn diagrams).

(b) Also check that

- not(A or B)
- (not A) and(not B)

are semantically equivalent.

(c) Semantically equivalent statements can still be different wrt. other aspects:

For instance, assume that evaluating a connective costs one time unit.

Which of the two semantically equivalent statements would then be "more efficient" wrt. time?

(d) These two semantic equivalences are also called "De Morgan's laws".

These "laws" are an immediate consequence of how we agreed to read the connectives not, and, or.

The quantifiers all (for-all, every) and some (exists, there is) are in some sense generalizations of and and or, respectively.

For instance, "All three persons, namely Alice, Bob, and Charlie, like ice cream" has the same meaning as "Alice likes ice cream and Bob likes ice cream and Charlie likes ice cream".

The negation "Not all three persons, namely Alice, Bob, and Charlie, like ice cream" thus is synonymous with "it is not true that: Alice likes ice cream and Bob likes ice cream and Charlie likes ice cream" resp. by De Morgan "Alice does not like ice cream or Bob does not like ice cream or Charlie does not like ice cream". The latter can be rephrased as "There is some person amongst Alice, Bob, and Charlie, who does not like ice cream".

In the standard way of reading the quantifiers all and some we therefore consider not all as synonymous with some not, and not some ("no", "none") as synonymous with all not. Example: (without highlighting the connectives and quantifiers)

- Recall that "All bandersnatches are borogoves" is short for "for all x it is true that: if x is a bandersnatch, then x is also a borogoves".

- Thus, "not all bandersnatches are borogoves" stands for "not for all x it is true that: if x is a bandersnatch, then x is also a borogoves".
- Pushing the negation inwards over the "for-all" we obtain: "there exists some x s.t. it is not true that: if x is a bandersnatch, then x is also a borogoves".
- By definition of "if-then": "there exists some x s.t. it is not true that: x is not a bandersnatch or x is also a borogoves".
- Pushing the negation over the "or" : "there exists some x s.t. it is true that: x is (not not) a bandersnatch and x is not also a borogoves".
- Short: "Some bandersnatches are not borogoves.".

Similarly argue that "no (short for: not some) bandersnatch is a borogoves" is sementically equivalent to "every bandersnatch is not a borogoves".

1.9 The Three Queens Puzzle (*)

In some sense, propositional logic is the most elementary programming language:

- Every CPU can be described by means of propositional logic.
- Every CPU (resp. its ALU) and every programming language understands not, and, or in the sense of propositional logic.
- Building a truth table is conceptually straight forward and every computer can do that for us (still, time consuming, possibly intractable).

In principle, we can use propositional logic to describe a problem resp. the *constraints* that define the problem, and then let the computer build a truth table for us in order to search for some (or all) potential solution(s).

A classical example is the n-Queens-Puzzle: you need to place n queens (the chess piece) on a $n \times n$ chess board in such a way, that no queen can take any other in a single move. There should be no additional pieces on the board.

Reminder of chess rules: A queen can take any chess piece that is in the same row (x-axis), same column (y-axis) or in one of her 45°-diagonals, given that no other piece is in between the two. So the problem amounts to placing the three queens in a way, that no two queens share a column, row or are in the same diagonal.

The problem becomes intractable for large n. For simplicity, we therefore only consider the case of $n = 3$. (Even then writing down the constraints becomes tedious.)

Let the fields of the chessboard be denoted by $(A,1), (A,2), (A,3), (B,1), \ldots, (C,3)$, where $(A,1)$ denotes the bottom left, $(A,3)$ the top left, $(C,1)$ the bottom right and $(C,3)$ the top right field, respectively.

Further let $D_{X,y}$ stand for the atomic proposition "A queen is on field (X,y)".

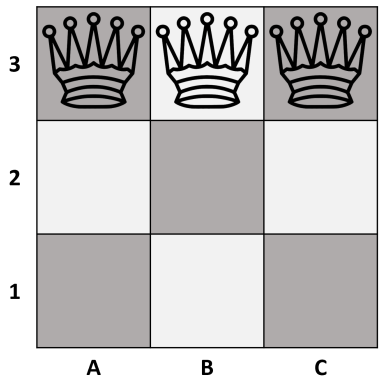
Example: The situation (obviously not a solution to the puzzle) where all three queens are in the top row would be represented by the assignment of the nine $D_{X,y}$-variables, where

$$D_{A,3} = D_{B,3} = D_{C,3} = \mathsf{true}$$

and

$$D_{A,1} = D_{B,1} = D_{C,1} = D_{A,2} = D_{B,2} = D_{C,2} = \mathsf{false}$$

Here's an illustration of the chess board this assignment of variables represents:



It is easy to imagine, that each assignment of the nine variables $D_{X,y}$ corresponds to exactly one such placement of queens on the board and vice versa.

Specify the following statements as propositional formulae using only the variables $D_{X,y}$, the connectors and, or, if-then, iff and not and possibly the statements already translated into logic before.

1. In each row there is (at least) one queen.
2. In each column there is (at least) one queen.
3. No two queens are in the same row.
4. No two queens are in the same column.
5. No two queens are on the same diagonal.
6. There at least one queen in each column and row and no two queens can take each other (i.e. essentially a formulation of the 3-Queens-Puzzle).